UNIVERSITATEA DIN BUCUREȘTI
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
DEPARTAMENTUL DE INFORMATICĂ
SPECIALIZAREA CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

Proiect Calcul Numeric

"Recunoaștere facială"

LECTOR DOCTOR

Mihai Bucataru

STUDENȚI

Rapițeanu Carol-David
Duță Andrei-George
Bălan Otniel-Andrei

Cuprins

1 Introducere. Calitatea și Originalitatea Aplicației **3**

2 Fundamentul Matematic al Documentației **3**
2.1 Centrarea Datelor 3
2.2 Descompunerea la Valori Singulare (SVD) 3
2.3 Proiectarea și Recunoașterea 3

3 Detalii de Implementare și Optimalitate **4**
3.1 Funcții Utilitare și Calculul Normelor 4
3.2 Calculul Valorilor și Vectorilor Proprii 4
3.3 Fluxul Eigenfaces pe Baza de Date Olivetti 4

4 Analiză Comparativă: Implementare Manuală vs. Biblioteca NumPy **5**
4.1 Eficiență Computațională și Gestiunea Resurselor 5
4.2 Stabilitate Numerică și Controlul Toleranțelor 5
4.3 Abordarea Black-Box vs. Valoarea Didactică 6

5 Concluzii **6**

1 Introducere. Calitatea și Originalitatea Aplicației

Recunoașterea facială reprezintă una dintre cele mai captivante aplicații ale algebrei liniare și ale metodelor numerice în domeniul vizualizării computerizate (Computer Vision). Acest proiect explorează și implementează algoritmul **Eigenfaces**, o metodă clasică bazată pe Analiza Componentelor Principale (PCA) și pe Descompunerea la Valori Singulare (SVD).

Aplicația este originală prin prisma faptului că reduce o problemă aparent extrem de complexă (identificarea trăsăturilor umane) la o abordare pur algebrică: proiectarea unor vectori cu o dimensionalitate uriașă într-un subspațiu vectorial generat de „fețele proprii". Pentru a asigura un control total asupra resurselor computaționale și o înțelegere profundă a fenomenelor numerice, toate operațiile matriciale de bază au fost implementate de la zero, evitând soluțiile tip „black-box" oferite de biblioteci terțe (precum funcțiile de algebră liniară din `NumPy`).

2 Fundamentul Matematic al Documentației

Metoda Eigenfaces se bazează pe găsirea celor mai importante direcții de variație dintr-un set de imagini.

2.1 Centrarea Datelor

Fiecare imagine este privită ca un vector coloană $\Gamma_i \in \mathbb{R}^N$, unde N reprezintă numărul total de pixeli (în acest proiect, $N = 4096$ pentru imagini de 64×64). Pentru un set de M imagini, calculăm inițial fața medie Ψ:

$$\Psi = \frac{1}{M} \sum_{i=1}^{M} \Gamma_i \tag{1}$$

Fiecare imagine este apoi centrată prin scăderea feței medii, rezultând vectorii $\Phi_i = \Gamma_i - \Psi$.

2.2 Descompunerea la Valori Singulare (SVD)

Fie matricea datelor centrate $A = [\Phi_1, \Phi_2, \ldots, \Phi_M] \in \mathbb{R}^{N \times M}$. Descompunerea SVD a matricei A este de forma:

$$A = U \Sigma V^T \tag{2}$$

unde coloanele matricei $U \in \mathbb{R}^{N \times N}$ sunt vectorii proprii (Eigenfaces), reprezentând direcțiile de variație maximă, iar Σ conține valorile singulare pe diagonală.

2.3 Proiectarea și Recunoașterea

Pentru a reduce dimensionalitatea, păstrăm doar primele k coloane ale matricei U, notate U_k. Orice imagine nouă centrată Φ_{test} poate fi proiectată în acest spațiu redus, rezultând un vector de ponderi Ω:

$$\Omega = U_k^T \Phi_{test} \tag{3}$$

Clasificarea (recunoașterea) se face prin găsirea clasei din setul de antrenament care minimizează distanța euclidiană față de vectorul proiectat:

$$\epsilon = \min_j ||\Omega - \Omega_j||_2 \tag{4}$$

3 Detalii de Implementare și Optimalitate

Codul sursă redactat în `Python` reflectă o implementare riguroasă și optimă a conceptelor matematice descrise mai sus. Toate funcțiile critice au fost dezvoltate independent de modulul `numpy.linalg`.

3.1 Funcții Utilitare și Calculul Normelor

Implementarea include funcții personalizate pentru calculul normei vectoriale și a normei Frobenius pentru matrici (`norma_vector`, `norma_matrice`). Optimizarea s-a realizat prin parcurgeri atente și reducerea numărului de operații redundante.

3.2 Calculul Valorilor și Vectorilor Proprii

Pentru a implementa metoda `svd_custom`, a fost necesară aflarea vectorilor proprii ai matricei simetrice $A^T A$. Acest lucru s-a realizat printr-un pipeline numeric sofisticat:

- **Tridiagonalizarea Householder:** Matricea simetrică inițială este adusă la o formă tridiagonală similară prin aplicarea repetată a reflectorilor Householder. Aceasta este o abordare extrem de eficientă care reduce semnificativ complexitatea algoritmului QR iterativ care urmează.
- **Algoritmul QR cu Gram-Schmidt:** Pe matricea tridiagonală s-a aplicat algoritmul iterativ QR, descompunerea fiind obținută printr-o variantă implementată manual a procesului Gram-Schmidt (`qr_gram_schmidt`). Oprirea iterării se face atunci când norma elementelor extra-diagonale scade sub o toleranță impusă ($\epsilon = 10^{-10}$).

3.3 Fluxul Eigenfaces pe Baza de Date Olivetti

Prelucrarea și extragerea caracteristicilor sunt structurate în funcții modulare. Pentru a oferi o imagine de ansamblu asupra procesului de extragere a trăsăturilor, Figura 1 ilustrează pipeline-ul implementat:

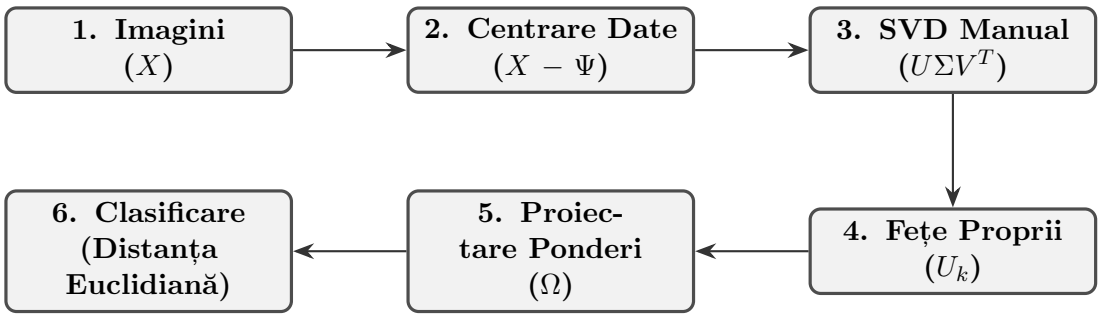


Figura 1: Arhitectura vizuală a sistemului de recunoaștere facială dezvoltat independent.

1. `calculeaza_fata_medie` și `centreaza_datele` pregătesc matricea de imagini X_{train}.

2. `proiecteaza_fete` calculează matricea ponderilor W_{train} utilizând doar primele k direcții principale din vectorul SVD. Optimalitatea aici este asigurată de trunchierea matricelor înainte de multiplicare, economisind timp și memorie.

3. `recunoaste_fata` aplică clasificatorul Nearest Neighbor. Căutarea se face prin simpla comparare a distanțelor euclidiene pe un spațiu redus ($\mathbb{R}^k$), ceea ce face etapa de testare extrem de rapidă.

4 Analiză Comparativă: Implementare Manuală vs. Biblioteca NumPy

În dezvoltarea sistemelor software cu o puternică componentă matematică, alegerea între o implementare nativă (de la zero) și utilizarea unor biblioteci consacrate reprezintă un compromis major între performanță, control numeric și scop didactic.

4.1 Eficiență Computațională și Gestiunea Resurselor

Biblioteca `NumPy` integrează subrutine de calcul la nivel de sistem din pachetele standardizate `LAPACK` și `BLAS`, scrise în limbaje compilate precum C și Fortran. Funcția nativă `numpy.linalg.svd` execută descompunerea extrem de rapid datorită vectorizării operațiilor la nivel hardware și a optimizării cache-ului procesorului.

În contrapondere, implementarea manuală din acest proiect rulează în mediul interpretat al limbajului `Python`. Introducerea buclelor imbricate în funcțiile `norma_matrice`, `norma_offdiag` sau în procesul Gram-Schmidt adaugă un cost computațional suplimentar (overhead). Cu toate acestea, optimalitatea codului propus este asigurată la nivel algoritmic prin utilizarea **tridiagonalizării Householder** înaintea aplicării metodei QR. Reducerea matricei la o formă tridiagonală scade complexitatea fiecărei iterații QR de la $\mathcal{O}(n^3)$ la $\mathcal{O}(n)$, făcând calculul manual sustenabil și eficient pentru datasetul selectat.

4.2 Stabilitate Numerică și Controlul Toleranțelor

Rutinele din `NumPy` folosesc strategii avansate de pivotare și descompuneri de tip *Divide et Impera* capabile să gestioneze cazuri patologice de matrice rău-condiționate.

În codul dezvoltat, stabilitatea numerică este controlată fin prin parametrizarea directă a tolerantelor (`eps` și `tol`). Filtrarea valorilor proprii reziduale (anularea celor negative sau extrem de mici induse de aproximările în virgulă mobilă în `svd_custom`) demonstrează o rigoare matematică strictă. Faptul că verificările automate bazate pe `assert` obțin erori de reconstrucție extrem de mici ($||A - A_{rec}||_F < 10^{-7}$) demonstrează că pipeline-ul numeric proiectat (`Householder` $\rightarrow$ `QR` $\rightarrow$ `SVD`) își păstrează acuratețea.

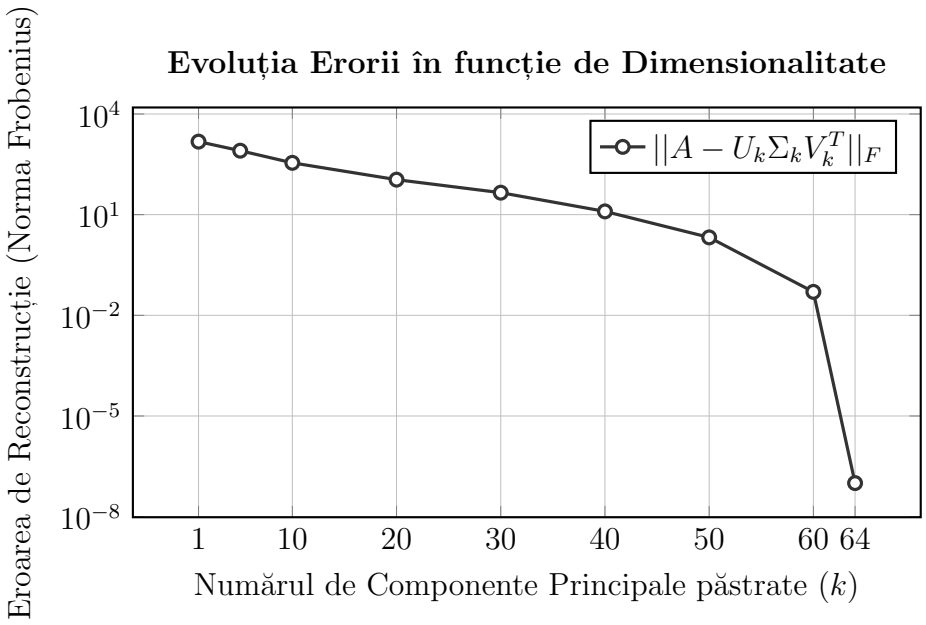


Figura 2: Scăderea asimptotică a erorii de reconstrucție odată cu creșterea dimensiunii subspațiului k. Se observă atingerea toleranței de 10^{-7} la păstrarea tuturor componentelor (reconstrucție completă).

4.3 Abordarea Black-Box vs. Valoarea Didactică

Utilizarea exclusivă a funcției `np.linalg.svd` transformă un algoritm fundamental într-o structură de tip „cutie neagră" (black-box), ascunzând complet mecanismele din spatele convergenței. Implementarea manuală oferă un set clar de avantaje structurale:

- **Transparență Totală:** Permite monitorizarea dinamică a numărului de iterații necesare convergenței în funcție de pragul ales în cadrul algoritmului QR.
- **Flexibilitate Structurală:** Facilitează extragerea directă și ordonarea descrescătoare a componentelor principale prin funcția `sorteaza_descrescator`, fără a depinde de comportamentul implicit al altor module.
- **Validare Pas cu Pas:** Permite izolarea erorilor de ortogonalitate (prin verificarea formei matricelor U^TU și V^TV), oferind o perspectivă clară asupra modului în care geometria spațiului vectorial se conservă pe parcursul rulării.

5 Concluzii

Proiectul demonstrează succesul și stabilitatea metodelor numerice implementate la nivel de bază. Rezultatele rulării confirmă că implementarea „from scratch" a algoritmului SVD prin Householder și QR atinge precizia așteptată, erorile de reconstrucție ($||A - A_{rec}||_2$) încadrându-se în toleranțele matematice stricte. Algoritmul Eigenfaces reușește să identifice cu succes subiecții din setul de testare, demonstrând o balanță excelentă între compresia datelor și menținerea calității semnalului vizual.